Atividade desenvolvida no estagio, projeto integrado ao banco de dados, aonde organizo as classes e metedodos

Classe Clientedata

```
public class Clientedata  //criando a classe Clientedata para acessar o banco de dados e realizar as operações de criar cliente, obter saldo e atualizar saldo
{
    private readonly string _construtorinicial; // string de conexão com o banco de dados

    1 referência
    public Clientedata(string construtor) // construtor da classe que recebe a string de conexão como parametro
    {
        _construtorinicial = construtor;
    }

    0 referências
    public double? ObterSaldo(int id) // metodo para obter o saldo do cliente a partir do id
    {
        using var conexao = new FbConnection(_construtorinicial);
        conexao.Open();
        string sql = "SELECT SALDO FROM CLIENTES WHERE ID = @id";    // aqui eu crio a string sql para selecionar o saldo da tabela aonde o ID for igual ao que o usuario digitou
        using var cmd = new FbCommand(sql, conexao);
        cmd.Parameters.AddWithValue("@id", id);
        var result = cmd.ExecuteScalar();
        return result != null ? Convert.ToDouble(result) : (double?)null; // retorna o saldo se encontrado, caso contrario retorna null
    }

    1 referência
    public (double Saldo, string Nome)? ObterCliente(int id)
    {
        using var conexao = new FbConnection(_construtorinicial);
        conexao.Open();
        string sql = "SELECT SALDO, NOME FROM CLIENTES WHERE ID = @id";
        using var cmd = new FbCommand(sql, conexao);
        cmd.Parameters.AddWithValue("@id", id);

        using var reader = cmd.ExecuteReader();
        if (reader.Read())
        {
            double saldo = reader.GetDouble(0);   // coluna SALDO
            string nome = reader.GetString(1);    // coluna NOME
            return (saldo, nome);
        }
        return null; // se não encontrar
    }


    1 referência
    public void CriarCliente(int id, double saldo, string nome)  // metodo para criar um novo cliente na tabela clientes
    {
        using var conexao = new FbConnection(_construtorinicial);
        conexao.Open();
        string sql = "INSERT INTO CLIENTES (ID, SALDO, NOME) VALUES (@id, @saldo, @nome) "; // inserir um novo cliente na tabela clientes com o id e saldo que foram passados como para
```

Aqui eu criei a classe Clientedata para centralizar todas as operações que conversam com o banco de dados. Logo no início, eu declarei um campo privado chamado _construtorinicial. Esse campo é do tipo string e está marcado como readonly, o que significa que ele só pode receber valor uma vez, no momento da criação do objeto, e depois não pode mais ser alterado. Isso garante que a string de conexão com o banco fique protegida e consistente durante toda a execução.

No construtor da classe, eu recebo uma string chamada construtor. Esse parâmetro é justamente a string de conexão que vem de fora, por exemplo, do Program.cs. Assim que o objeto é criado, eu atribuo esse valor ao campo _construtorinicial. A partir daí, todos os métodos da classe podem usar _construtorinicial para abrir conexões com o banco sem precisar receber novamente a string. É como se eu guardasse a chave de acesso ao banco dentro da classe, e cada método que precisar abrir a porta vai usar essa mesma chave.

Classe Contabancaria

```
Contabancaria.cs   Program.cs*
ContaBancaria                                   Sacar(double valor, Clientedata repo)
4 referências
public class ContaBancaria
{
    5 referências
    public int Id { get; private set; } // aqui eu criei a propriedade Id para armazenar o id do cliente, assim eu posso usar o id para acessar o banco de dados
    12 referências
    public double Saldo { get; private set; } // aqui eu criei a propriedade Saldo para armazenar o saldo do cliente, assim eu posso mostrar o saldo do cliente
    1 referência
    public string Nome { get; private set; } // aqui eu criei a propriedade Nome para armazenar o nome do cliente, assim eu posso mostrar o nome do cliente
    2 referências
    public ContaBancaria(int id, double saldo, string nome)
    {

        Id = id; // aqui eu criei o construtor da classe ContaBancaria que recebe o id, saldo e nome do cliente como parametros e atribui esses valores as propriedades da classe
        Saldo = saldo;
        Nome = nome;
    }

    1 referência
    public void ConsultarSaldo() // metodo consultar saldo, aqui eu só mostro o saldo atual do cliente formatado com 2 casas decimais
    {
        Console.WriteLine($"Seu saldo atual é: R$ {Saldo:F2}");
    }

    1 referência
    public void Sacar(double valor, Clientedata repo) // metodo sacar verificando se o valor é menor ou igual ao saldo disponivel

    {
        if (valor <= Saldo)
        {
            Saldo -= valor;
            repo.AtualizarSaldo(Id, Saldo); // atualiza o saldo no repositório de dados do cliente
            Console.WriteLine("Saque realizado.");
        }
        else
        {
            Console.WriteLine("Saldo insuficiente!");

        }
    }


    1 referência
    public void Depositar(double valor, Clientedata repo) // metodo depositar, aqui eu só somo o valor do deposito com o saldo atual e mostro uma mensagem de sucesso
    {
        Saldo += valor;
        repo.AtualizarSaldo(Id, Saldo);
        Console.WriteLine("Depósito realizado com sucesso!");
    }
```

Eu desenvolvi essa classe chamada **ContaBancaria** para representar o funcionamento de uma conta dentro de um sistema bancário. A ideia foi criar uma estrutura que não apenas guardasse informações como o id, o saldo e o nome do cliente, mas que também permitisse realizar operações reais, como consultar o saldo, sacar, depositar e até solicitar um empréstimo.

O que eu quis mostrar é que cada operação altera o estado da conta e, ao mesmo tempo, mantém o banco de dados atualizado. Por exemplo, quando o cliente faz um saque ou um depósito, o saldo é ajustado e imediatamente salvo no repositório, garantindo que os dados fiquem consistentes. No caso do empréstimo, eu estabeleci uma regra: só é possível solicitar se o saldo for maior ou igual a cinco mil. Isso traz uma lógica de negócio simples, mas que já demonstra como podemos aplicar condições reais em um sistema.

O objetivo dessa classe é ser a ponte entre o usuário e o banco de dados, permitindo que as ações feitas no programa tenham efeito direto e persistente. Com isso, eu consigo simular de forma prática como funcionaria um sistema

bancário básico, em que cada decisão do cliente tem impacto imediato e transparente.

Program.cs

```
Contabancaria.cs    Program.cs*
Estagio.Programa    Main(string[] args)
using FirebirdSql.Data.FirebirdClient;
using System;

namespace Estagio
{
    0 referências
    class Programa
    {
        0 referências
        static void Main(string[] args)
        {
            string connString = "User=SYSDBA;Password=masterkey;Database=C:\\Pasta de Trabalho\\SAR\\SAR\\bin\\packed\\scripts\\Meubanco\\A.fdb;DataSource=localhost;Port=4050;";
            var repo = new Clientedata(connString);

            Console.WriteLine("Bem-vindo ao sistema bancário!");
            Console.WriteLine("\nPossui ID de acesso? (sim/nao)");
            string resposta2 = Console.ReadLine().ToLower(); // esperando a resposta do cliente

            int id; // variavel para armazenar o id do cliente
            ContaBancaria conta; // variavel para armazenar a conta

            if (resposta2 == "sim")
            {
                Console.Write("Digite seu ID: ");
                if (int.TryParse(Console.ReadLine(), out id)) // validação para garantir que o ID seja um número inteiro
                {
                    var cliente = repo.ObterCliente(id); // método que retorna saldo + nome
                    if (cliente != null) // validação para garantir que o ID exista no banco de dados
                    {
                        conta = new ContaBancaria(id, cliente.Value.Saldo, cliente.Value.Nome); // criação da conta do cliente com os dados obtidos do banco
                        Console.WriteLine($"Bem-vindo de volta, cliente {cliente.Value.Nome}! Seu saldo atual é de R$ {cliente.Value.Saldo:F2}");
                    }
                    else
                    {
                        Console.WriteLine("ID não encontrado no banco, sistema será fechado contra fraudes.");
                        return;
                    }
                }
                else
                {
                    Console.WriteLine("ID inválido.");
                    return;
                }
            }
            else // fluxo para criar cliente novo
            {
                Console.Write("Digite o ID que foi informado pelo caixa eletrônico: ");
                if (int.TryParse(Console.ReadLine(), out id))
                {
                    Console.Write("\nAgora digite seu nome: ");
                    string nome = Console.ReadLine();
```

Esse código funciona como o coração do programa, e o fluxo dele pode ser entendido passo a passo. Primeiro, eu estabeleço a conexão com o banco de dados Firebird, criando a string de conexão e instanciando a classe Clientedata, que é responsável por acessar e manipular os dados dos clientes. Logo em seguida, o sistema dá as boas-vindas e pergunta se o usuário já possui um ID de acesso.

Se a resposta for “sim”, o programa solicita o ID e valida se é um número inteiro. Caso seja válido, ele consulta o banco com o método ObterCliente. Se o cliente existir, cria uma instância da conta com os dados retornados e mostra uma mensagem personalizada com o nome e o saldo atual. Se o ID não for encontrado, o programa encerra para evitar fraudes. Se o ID digitado não for válido, também encerra.

Se a resposta inicial for “não”, o fluxo muda para a criação de um novo cliente. Nesse caso, o programa pede o ID e o nome, gera um saldo inicial aleatório entre 100 e 5000, grava esses dados no banco com o método CriarCliente e

cria a conta em memória. Depois, mostra uma mensagem confirmando o registro e exibindo o saldo inicial.

Com a conta criada ou carregada, o programa entra em um laço de repetição que apresenta um menu de operações: consultar saldo, sacar, depositar, solicitar empréstimo ou sair. Cada opção chama o método correspondente da classe ContaBancaria. O saque e o depósito atualizam o saldo tanto na memória quanto no banco. O empréstimo só é concedido se o saldo for maior ou igual a cinco mil, e nesse caso adiciona dez mil ao saldo. Ao escolher sair, o programa garante que o saldo final seja persistido no banco antes de encerrar.

Dessa forma, o código integra interação com o usuário, regras de negócio e persistência de dados, simulando de maneira prática como funcionaria um sistema bancário básico.